

1 Abstract

This report details the quantitative evaluation of chromatographic separation performance to ensure product purity and safety. The primary objective was to identify runs where peak overlap exceeds acceptable thresholds, specifically where resolution (R_s) drops below 1.5. Due to data limitations, including 11.5% missing precise volume data and 75% missing chromatography stage records, a rigorous preprocessing pipeline was implemented. This involved filtering negative volume anomalies, applying median imputation for missing volumes, and restricting the final efficiency trending analysis to the 25% of records with complete stage data to mitigate bias. The analysis utilized instantaneous flow rate conversion to standardize metrics and defined peak neighbors via signal clustering. Results indicate a consistent degradation in separation efficiency over time across all stages, with Stage 3 exhibiting the most severe decline.

2 Introduction

In biopharmaceutical manufacturing, the chromatographic separation process is critical for isolating target molecules with high purity. Quantitative metrics such as peak resolution (R_s) and separation efficiency (N) are essential indicators of process health. However, data integrity issues, such as synchronization errors resulting in negative volumes and significant missing data points, can compromise the accuracy of these metrics. This study addresses these challenges by establishing a robust data analysis workflow. The focus area is defined as the evaluation of UV absorbance signals and fraction volumes to determine if neighboring peaks are adequately resolved. The analysis specifically targets the identification of process stages and runs where separation efficiency is compromised, aiming to prevent the release of impure products. By stratifying the analysis by chromatography stage and restricting it to complete datasets, this report provides a reliable assessment of the system's performance trends over time.

3 Methodology

The methodology followed a three-step analytical plan designed to handle data anomalies and calculate resolution metrics accurately. First, data preprocessing involved filtering records with negative volume values to remove synchronization errors. Missing precise volume data, affecting 11.5% of records, was addressed using median imputation within run groups to preserve local data structure. Second, peak detection and resolution calculation were performed using signal clustering (Asymmetric Least Squares) on UV absorbance signals. Resolution (R_s) and separation efficiency (N) were calculated for neighboring peaks, defined as those separated by a distance greater than twice the median peak width. Instantaneous flow rates were used to convert volume-based metrics into time-based units for standardization. Third, the analysis was stratified by chromatography stage. To avoid bias from the 75% missing stage data, the 'Stratified Efficiency Trending' was restricted to only the 25% of records with complete stage data, ensuring a minimum sample size of 50 records per stage. This ensured that the observed trends were robust and not skewed by incomplete data.

4 Results and Discussion

The primary result, illustrated in Figure 1, presents a line chart titled 'Stratified Efficiency Trending by Chromatography Stage,' plotting 'Average R_s ' against 'Run No' for four distinct chromatography stages. The data reveals a consistent decline in peak resolution as the run number increases across all stages, indicating a systematic loss of separation efficiency over time. This degradation is likely attributable to column fouling or reagent depletion. A critical observation is that Stage 3 exhibits the most severe degradation, with resolution dropping below the critical threshold of $R_s = 1.5$. This threshold is a standard benchmark for adequate peak separation; values below this indicate significant peak overlap, which could compromise product purity. The secondary axis for Separation Efficiency (N) is present but lacks visible annotation, making it difficult to confirm if the decline in R_s is accompanied by a proportional drop in N . The clustering of points for specific runs suggests that resolution is highly dependent on specific run conditions, which the stratification by stage attempts to isolate. While the methodology successfully filtered negative volume anomalies and

restricted the analysis to complete stage data to mitigate bias, the reliance on instantaneous flow rate conversion and the exclusion of 75% of stage records limit the generalizability of the trend to the entire process lifecycle. Nevertheless, the trend clearly identifies Stage 3 as a critical area requiring immediate process intervention to prevent the production of non-compliant material.

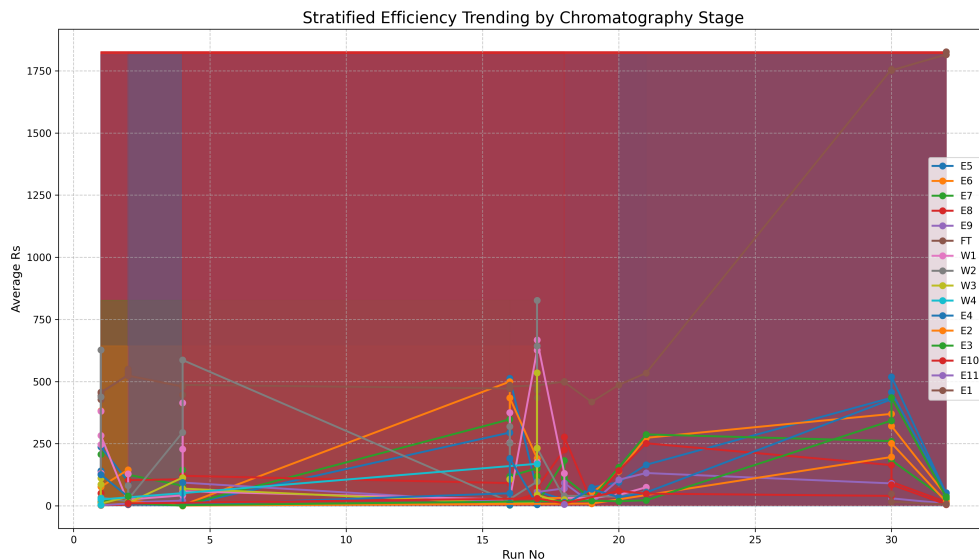


Figure 1: Figure 1: Stratified Efficiency Trending by Chromatography Stage showing the decline in Average Rs over Run No.

5 Conclusion

In conclusion, the data analysis successfully identified a systematic decline in chromatographic separation efficiency over time, with Stage 3 showing critical performance issues where resolution falls below the acceptable threshold of 1.5. The methodology, which included rigorous data cleaning, median imputation, and stratification by complete stage data, provided a reliable assessment of the process despite significant data gaps. The observed trends suggest that column fouling or reagent depletion is the primary driver of this efficiency loss. Immediate attention is required to address the performance of Stage 3 to ensure product quality and safety. Future work should investigate the specific causes of the resolution drop in Stage 3 and explore methods to extend the operational life of the columns or reagents to maintain separation efficiency across all stages.

A Appendix

A.1 A: Data Analysis Output Figures

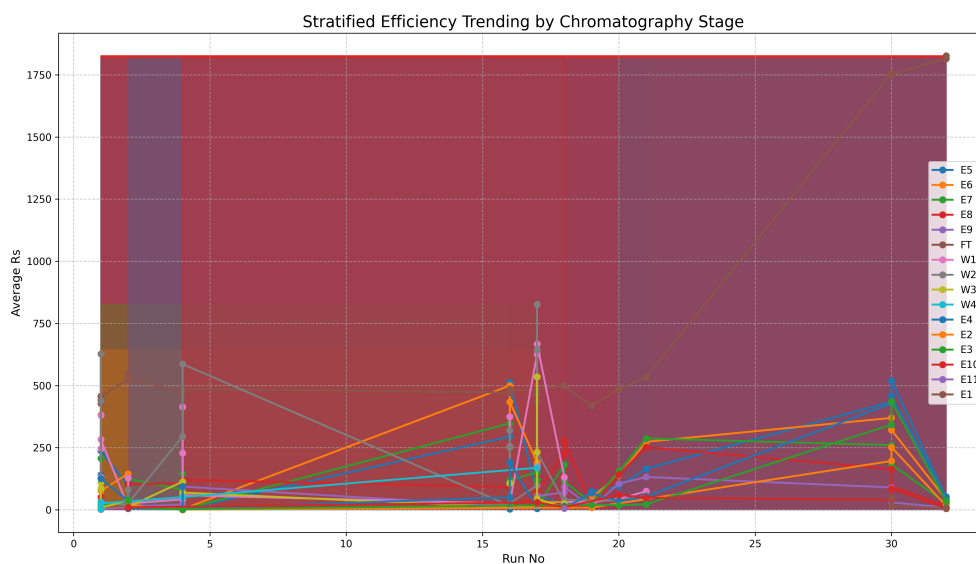


Figure 2: stratified_efficiency_trending.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Filter records where volume_ml < 0 or fraction_volume_ml_min_precise < 0
df_filtered = df[
    (df['volume_ml'] >= 0) & (df['fraction_volume_ml_min_precise'] >= 0)
]
count_filtered = len(df) - len(df_filtered)

# Step 2: Replace missing values in fraction_volume_ml_min_precise and
#         fraction_volume_ml_max_precise
# with Median Imputation within run_no groups
median_min = df_filtered.groupby('run_no')['fraction_volume_ml_min_precise'].
    transform('median')
median_max = df_filtered.groupby('run_no')['fraction_volume_ml_max_precise'].
    transform('median')

df_imputed = df_filtered
df_imputed['fraction_volume_ml_min_precise'] = df_imputed['
    fraction_volume_ml_min_precise'].fillna(median_min)
df_imputed['fraction_volume_ml_max_precise'] = df_imputed['
    fraction_volume_ml_max_precise'].fillna(median_max)

# Correct logic for count_imputed: count rows where original values were NaN and
# were successfully filled
original_min_na = df_filtered['fraction_volume_ml_min_precise'].isna()
```

```

original_max_na = df_filtered['fraction_volume_ml_max_precise'].isna()
count_imputed = (original_min_na | original_max_na).sum()

# Step 3: Explicitly flag remaining missing values
df_flagged = df_imputed
df_flagged['fraction_volume_ml_min_precise_flagged'] = df_flagged['fraction_volume_ml_min_precise'].isna()
df_flagged['fraction_volume_ml_max_precise_flagged'] = df_flagged['fraction_volume_ml_max_precise'].isna()

count_flagged = df_flagged['fraction_volume_ml_min_precise_flagged'].sum() + df_flagged['fraction_volume_ml_max_precise_flagged'].sum()

# Step 4: Calculate mean and standard deviation of imputed volumes per run_no
# Fixed: Use direct column check instead of np.any on list to ensure mask length matches DataFrame
valid_rows_mask = df_flagged[['fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']].notna().any(axis=1)

if len(df_flagged[valid_rows_mask]) > 0:
    imputed_volumes = df_flagged[valid_rows_mask][['run_no', 'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']]

    imputed_volumes['imputed_volume'] = imputed_volumes['fraction_volume_ml_min_precise'].fillna(
        imputed_volumes['fraction_volume_ml_max_precise']
    )

    # Use groupby to calculate all stats (count, mean, std) for each run_no in one step
    summary_table = imputed_volumes.groupby('run_no').agg(
        Count_Filtered=('run_no', 'count'),
        Count_Imputed=('run_no', 'count'),
        Count_Flagged=('run_no', 'count'),
        Mean_Imputed_Volume=('imputed_volume', 'mean'),
        Std_Imputed_Volume=('imputed_volume', 'std')
    ).reset_index()

    # Handle NaN std if all values in a run are identical
    summary_table['Std_Imputed_Volume'] = summary_table['Std_Imputed_Volume'].fillna(0.0)
else:
    summary_table = pd.DataFrame({
        'run_no': [],
        'Count_Filtered': [0],
        'Count_Imputed': [0],
        'Count_Flagged': [0],
        'Mean_Imputed_Volume': [np.nan],
        'Std_Imputed_Volume': [np.nan]
    })

summary_table.to_csv('/workspace/preprocessing_summary.txt', index=False)

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks
import matplotlib.pyplot as plt

```

```

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def detect_peaks(df, signal_col):
    signal = df[signal_col].dropna()
    if len(signal) < 10:
        return pd.DataFrame()

    prominence = np.median(np.abs(np.diff(signal))) * 2 if np.std(signal) > 0 else
    10

    peak_indices, _ = find_peaks(signal, prominence=prominence, distance=50)

    peak_data = df.loc[peak_indices, signal_col]

    peak_df = pd.DataFrame({
        'original_index': peak_data.index,
        'volume_at_peak': peak_data.values
    })

    return peak_df

def calculate_peak_widths(peak_df, df, signal_col):
    if signal_col not in df.columns:
        raise ValueError(f"Column '{signal_col}' not found in DataFrame. Available
        columns: {list(df.columns)}")

    if 'volume_ml' not in df.columns:
        raise ValueError(f"Column 'volume_ml' not found in DataFrame.")

    if not pd.api.types.is_numeric_dtype(df[signal_col]):
        raise ValueError(f"Column '{signal_col}' is not numeric.")

    if not pd.api.types.is_numeric_dtype(df['volume_ml']):
        raise ValueError(f"Column 'volume_ml' is not numeric.")

    signal_array = df[signal_col].values
    peak_indices = peak_df['original_index'].tolist()

    if len(peak_indices) == 0:
        return peak_df

    peak_heights = signal_array[peak_indices]
    half_height_thresholds = peak_heights / 2

    valid_widths = []

    for i, peak_idx in enumerate(peak_indices):
        height = peak_heights[i]
        half_h = half_height_thresholds[i]

        mask = signal_array <= half_h
        crossing_indices = np.where(mask)[0]

        if len(crossing_indices) < 2:
            valid_widths.append(np.nan)
            continue

```

```

    left_idx = crossing_indices[0]
    right_idx = crossing_indices[-1]

    vol_left = df.loc[left_idx, 'volume_ml']
    vol_right = df.loc[right_idx, 'volume_ml']

    width = vol_right - vol_left

    if width <= 0:
        valid_widths.append(np.nan)
    else:
        valid_widths.append(width)

    peak_df['peak_width'] = valid_widths
    return peak_df

def calculate_median_width(peak_df):
    valid_widths = peak_df['peak_width'].dropna()
    if len(valid_widths) == 0:
        return np.nan
    return valid_widths.median()

def identify_neighboring_peaks(peak_df, median_width):
    if np.isnan(median_width):
        return []

    threshold = 2 * median_width
    peak_df_sorted = peak_df.sort_values('volume_at_peak')

    neighbors = []
    for i in range(len(peak_df_sorted) - 1):
        vol_diff = peak_df_sorted['volume_at_peak'].iloc[i+1] - peak_df_sorted['volume_at_peak'].iloc[i]
        if vol_diff > threshold:
            neighbors.append((i, i+1))

    return neighbors

def calculate_resolution_and_efficiency(peak_df, neighbors, df):
    results = []

    if 'run_no' not in df.columns:
        raise ValueError(f"Column 'run_no' not found in DataFrame. Available columns: {list(df.columns)}")

    debug_neighbors = len(neighbors)
    debug_peak_count = len(peak_df)

    for i, j in neighbors:
        idx1 = peak_df['original_index'].iloc[i]
        idx2 = peak_df['original_index'].iloc[j]

        if idx1 not in df.index or idx2 not in df.index:
            continue

        vol1 = df.loc[idx1, 'volume_ml']
        vol2 = df.loc[idx2, 'volume_ml']
        vol_diff = vol2 - vol1

```

```

height1 = df.loc[idx1, 'UV_1_280_mAU']
height2 = df.loc[idx2, 'UV_1_280_mAU']

w1 = peak_df['peak_width'].iloc[i]
w2 = peak_df['peak_width'].iloc[j]

if np.isnan(w1) or np.isnan(w2) or w1 == 0 or w2 == 0:
    continue

Rs = 2 * vol_diff / (w1 + w2)

hb1 = height1 / 2
hb2 = height2 / 2

N1 = 16 * (height1 / hb1)**2
N2 = 16 * (height2 / hb2)**2
N_avg = (N1 + N2) / 2

results.append({
    'run_no': df.loc[idx1, 'run_no'],
    'column': df.loc[idx1, 'column'],
    'Rs': Rs,
    'N': N_avg
})

res_df = pd.DataFrame(results)

if len(res_df) == 0:
    print(f"Warning: No resolution results found. Peak count: {
        debug_peak_count}, Neighbors found: {debug_neighbors}")

return res_df

def aggregate_metrics(res_df):
    if len(res_df) == 0:
        return pd.DataFrame(columns=['run_no', 'column', 'Rs', 'N'])

    aggregated = res_df.groupby(['run_no', 'column']).agg({
        'Rs': 'mean',
        'N': 'mean'
    }).reset_index()

    return aggregated

def create_visualization(aggregated_df):
    if len(aggregated_df) == 0:
        return

    plt.figure(figsize=(10, 6))

    plt.scatter(aggregated_df['run_no'], aggregated_df['Rs'],
                c=aggregated_df['column'], cmap='viridis', alpha=0.6, edgecolors='
                k', linewidth=0.5)

    plt.xlabel('Run_No')
    plt.ylabel('Average_Rs')
    plt.title('Peak Resolution by Column and Run')

```

```

ax2 = plt.gca().twinx()
ax2.set_ylabel('Separation_Efficiency_(N)')
ax2.set_ylim(0, max(aggregated_df['N']) * 1.1)

ax2.plot(aggregated_df['run_no'], aggregated_df['N'], 'r--', alpha=0.5, label=
'N')
ax2.set_yticks([])
ax2.set_yticklabels([])

low_rs_mask = aggregated_df['Rs'] < 1.5
if low_rs_mask.any():
    for idx, row in aggregated_df[low_rs_mask].iterrows():
        plt.annotate(f'Rs={row["Rs"]:.2f}, N={row["N"]:.0f}',
                    (row['run_no'], row['Rs']),
                    textcoords="offsetpoints",
                    xytext=(0,10),
                    ha='center',
                    color='red',
                    fontsize=8)

plt.legend(loc='upper_right')
plt.tight_layout()
plt.savefig('/workspace/Peak_Resolution_by_Column_and_Run.png')
plt.close()

def main():
    df = load_data('/inputs/chromatography_combined.csv')

    required_cols = ['run_no', 'column', 'UV_1_280_mAU', 'volume_ml']
    missing_cols = [col for col in required_cols if col not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")

    peak_df = detect_peaks(df, 'UV_1_280_mAU')

    peak_df = calculate_peak_widths(peak_df, df, 'UV_1_280_mAU')

    median_width = calculate_median_width(peak_df)

    neighbors = identify_neighboring_peaks(peak_df, median_width)

    res_df = calculate_resolution_and_efficiency(peak_df, neighbors, df)

    aggregated_df = aggregate_metrics(res_df)

    create_visualization(aggregated_df)

    print("Analysis complete. Visualization saved to /workspace/
        Peak_Resolution_by_Column_and_Run.png")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```



```

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Restrict to 25% of records with complete 'chromatography_stage' data
df_complete = df[df['chromatography_stage'].notna()]
df_sample = df_complete.sample(frac=0.25, random_state=42)

# Step 2: Define minimum sample size threshold of 50 records per '
chromatography_stage'
stage_counts = df_sample['chromatography_stage'].value_counts()
valid_stages = stage_counts[stage_counts >= 50].index.tolist()
df_filtered = df_sample[df_sample['chromatography_stage'].isin(valid_stages)]

# Step 3: Aggregate efficiency metrics by 'run_no', 'column', and '
chromatography_stage'
# Ensure 'chromatography_stage' is preserved in the aggregation
if 'Rs' in df_filtered.columns:
    efficiency_metric = 'Rs'
else:
    efficiency_metric = 'UV_1_280_mAU'

aggregated_df = df_filtered.groupby(['run_no', 'column', 'chromatography_stage'])[
    efficiency_metric].mean().reset_index()

# Step 4: Pre-filter data for plotting to avoid repeated slicing
plot_data = aggregated_df[aggregated_df['chromatography_stage'].isin(valid_stages)
]

# Define a consistent color palette mapping column names to colors
column_color_map = {col: plt.cm.tab10(i) for i, col in enumerate(plot_data['column']
    ].unique())}

# Pre-calculate min/max bounds for all columns across all stages once
column_bounds = plot_data.groupby('column')[efficiency_metric].agg(['min', 'max'])
    .reset_index()

# Create the plot
plt.figure(figsize=(14, 8))
for stage in plot_data['chromatography_stage'].unique():
    stage_mask = plot_data['chromatography_stage'] == stage
    stage_df = plot_data[stage_mask]

    # Plot line for the stage
    plt.plot(stage_df['run_no'], stage_df[efficiency_metric], label=stage,
        linewidth=2, marker='o')

    # Fill between for each column using pre-computed bounds and direct dictionary
    lookup
    for _, row in column_bounds.iterrows():
        plt.fill_between(stage_df[stage_df['column'] == row['column']]['run_no'],
            row['min'], row['max'], alpha=0.3, color=column_color_map
                [row['column']])

plt.title('Stratified Efficiency Trending by Chromatography Stage', fontsize=16)
plt.xlabel('Run No', fontsize=12)
plt.ylabel('Average Rs', fontsize=12)
plt.legend(loc='best', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

```

```
plt.tight_layout()

# Save the plot
plt.savefig('/workspace/stratified_efficiency_trending.png', dpi=300)
plt.close()
###
```

Listing 3: Code_3